Recruiter Job Postings (US-12 and US-13)

Purpose of AI Use

Requirements clarification, implementation planning, code generation, debugging, and validation for recruiter job posting creation and management.

AI-Suggested Content

The AI reviewed issues #17 (Recruiter Job Posting Creation) and #18 (Recruiter Job Posting Management) and helped implement the requested feature. Job seekers can view postings but cannot create or manage them. Applying to a posting was left out of scope.

Validation

The AI ran `pnpm check`, `pnpm test`, and `pnpm build`; all completed successfully. The test suite passed, including job posting validation tests. During manual testing, the user confirmed that job creation and posting management worked after the form action and database table issues were corrected.

Decision

Accepted / Modified. The implementation followed existing SvelteKit and Drizzle patterns. Manual testing led to a correction for SvelteKit's reserved action name and a narrowly scoped database schema update.

Reflection

AI helped turn two related user stories into a working end-to-end feature. Server-side authorization was necessary in addition to hiding management controls in the interface: create checks the account role, while edit and delete also scope database operations to the posting owner. The database error demonstrated the value of checking the configured database schema rather than assuming that local schema changes had already been applied.

Responsible Person

Michael Derocher

Prompt 1

I need to make a page for creating job postings, which should be saved to our database and viewable to other people. Only recruiter accounts should create postings; job seekers should be able to view them. The recruiter who created a posting should be able to edit and delete it. Job seekers do not need to apply yet. The related issues are #17 and #18. Do you understand the task?

Response 1

Reviewed the acceptance criteria in both issues:

- **#17, US-12:** recruiter can create a job posting; required fields must be completed; job seekers can see the posting; recruiter receives confirmation.
- **#18, US-13:** recruiter can update and delete their postings; changes appear immediately; unauthorized users cannot modify postings.

Proposed the implementation approach: add a posting table to the existing Drizzle schema; validate submitted fields on the server; add job listing/detail and recruiter create/edit pages; and enforce role and ownership checks in the server actions rather than relying on UI visibility alone. Asked whether viewing should be restricted to signed-in users or open to visitors.

Prompt 2

Only people who are signed in should be able to view it. Please go ahead and implement your suggestions.

Response 2

Implemented the authenticated job posting feature. The app's existing server hook already redirects signed-out visitors to `/login`, so the new `/jobs` routes remain signed-in-only without changing the public route allowlist.

Files modified or added for the feature

- `README.md` — documented `pnpm db:push` for syncing development schema changes.
- `src/lib/server/db/schema.ts` — added the `jobPosting` table and exported `JobPosting`.
- `src/lib/server/job-posting.ts` — added server-side parsing and validation.
- `src/lib/server/job-posting.test.ts` — added tests for trimming, required fields, invalid values, inherited object property names, and description length.
- `src/lib/components/JobPostingForm.svelte` — added shared create/edit fields, inline errors, pending state, and delete confirmation.
- `src/routes/(app)/+layout.svelte` — added Jobs navigation and nested-route active state.
- `src/routes/(app)/+page.svelte` — added links from home to browse postings or create one.
- `src/routes/(app)/jobs/+page.server.ts` and `src/routes/(app)/jobs/+page.svelte` — load and display postings newest first; show create links to recruiters.
- `src/routes/(app)/jobs/new/+page.server.ts` and `src/routes/(app)/jobs/new/+page.svelte` — enforce recruiter-only access and provide the create form/action.
- `src/routes/(app)/jobs/[id]/+page.server.ts` and `src/routes/(app)/jobs/[id]/+page.svelte` — load and display posting details to signed-in users; show the edit link only to the creator.
- `src/routes/(app)/jobs/[id]/edit/+page.server.ts` and `src/routes/(app)/jobs/[id]/edit/+page.svelte` — provide owner-scoped edit and delete operations.

The database schema added a UUID posting ID, required creator foreign key (cascading on account deletion), required title/company/location/employment type/work mode/description fields, creation and update timestamps, and indexes on creator and creation time. Representative code from `src/lib/server/db/schema.ts`:

```
export const jobPosting = pgTable(
    'job_posting',
    {
        id: uuid('id').primaryKey().defaultRandom(),
        createdBy: text('created_by')
            .notNull()
            .references(() => user.id, { onDelete: 'cascade' }),
        title: text('title').notNull(),
        company: text('company').notNull(),
        location: text('location').notNull(),
        employmentType: text('employment_type').notNull(),
        workMode: text('work_mode').notNull(),
        description: text('description').notNull(),
        createdAt: timestamp('created_at').defaultNow().notNull(),
        updatedAt: timestamp('updated_at').defaultNow().notNull()
    },
    (t) => [index('job_posting_created_by_idx').on(t.createdBy), index('job_posting_created_at_i
);
```

`src/lib/server/job-posting.ts` validates the submitted `FormData`: title, company, and location are trimmed, required, and limited to the existing 100-character `LIMITS.short`; employment type and work mode must be own keys of the existing choice maps; description is required and capped at 5000 characters. `Object.hasOwn` rejects inherited names such as `toString` and `constructor`:

```
if (!Object.hasOwn(EMPLOYMENT_TYPES, values.employmentType)) {
    errors.employmentType = 'Choose a valid employment type.';
}
if (!Object.hasOwn(WORK_MODES, values.workMode)) errors.workMode = 'Choose a valid work mode.';
if (!values.description) errors.description = 'Description is required.';
else if (values.description.length > DESCRIPTION_LIMIT) {
    errors.description = `Description must be ${DESCRIPTION_LIMIT} characters or fewer.`;
}
```

The create server action in `src/routes/(app)/jobs/new/+page.server.ts` checks recruiter role, validates the submitted fields, and assigns ownership from `locals.user.id` rather than accepting an owner ID from the client:

```
if (locals.user!.role !== 'recruiter') {
    return fail(403, { message: 'Only recruiters can create job postings.' });
}
const parsed = parseJobPosting(await request.formData());
if (!parsed.ok) return fail(400, { errors: parsed.errors, values: parsed.values });
const [posting] = await db
    .insert(jobPosting)
    .values({ ...parsed.values, createdBy: locals.user!.id })
    .returning({ id: jobPosting.id });
redirect(303, `/jobs/${posting.id}`);
```

The edit and delete operations in `src/routes/(app)/jobs/[id]/edit/+page.server.ts` require recruiter role and scope each database operation to both the posting ID and logged-in user ID:

```
.where(and(eq(jobPosting.id, params.id), eq(jobPosting.createdBy, locals.user!.id)))
```

The detail page returns an `isOwner` flag for rendering the edit link, while the server actions independently enforce ownership. Job seekers can browse and view postings, but cannot create, edit, or delete them.

Validation after implementation: `pnpm check`, `pnpm test`, and `pnpm build` all passed.

Prompt 3

When creating a posting, SvelteKit returned `Cannot use reserved action name "default"`.

Response 3

Fixed the form action name in two files:

- `src/routes/(app)/jobs/new/+page.server.ts` — renamed the action key from `default` to `create`.
- `src/routes/(app)/jobs/new/+page.svelte` — changed the form target from `?/default` to `?/create`.

The form now submits to the named SvelteKit action:

```
export const actions: Actions = {
    create: async ({ request, locals }) => {
        // role check, validation, insert, and redirect
    }
};
```

```
<JobPostingForm action="?/create" submitLabel="Publish posting" values={form?.values ?? {}} erro
```

`pnpm check` passed after this correction.

Prompt 4

Creating a posting then failed with a database insert error.

Response 4

Inspected the configured database and found that `public.job_posting` did not exist, explaining the failed insert. Tried `pnpm db:push`; it could not proceed in the non-interactive terminal and displayed additional proposed changes to unrelated existing constraints, defaults, and row-level security settings. Those unrelated changes were not applied.

Applied only the required posting table, creator foreign key, and indexes. The SQL applied to the configured database was:

```
CREATE TABLE public.job_posting (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid() NOT NULL,
    created_by text NOT NULL,
    title text NOT NULL,
    company text NOT NULL,
    location text NOT NULL,
```

```
    employment_type text NOT NULL,
    work_mode text NOT NULL,
    description text NOT NULL,
    created_at timestamp DEFAULT now() NOT NULL,
    updated_at timestamp DEFAULT now() NOT NULL,
    CONSTRAINT job_posting_created_by_user_id_fk
        FOREIGN KEY (created_by) REFERENCES public."user"(id) ON DELETE CASCADE
);
CREATE INDEX job_posting_created_by_idx ON public.job_posting USING btree (created_by);
CREATE INDEX job_posting_created_at_idx ON public.job_posting USING btree (created_at);
```

Queried the database after applying the SQL and verified that the table had all 10 expected columns and both indexes. The user then confirmed that posting creation worked.